**UNIVERSIDAD DEL VALLE DE GUATEMALA**

*DEEP LEARNING - 10*

LUIS ALBERTO SURIANO SARAVIA




# Entrega parcial del proyecto final

Ihan Marroquin - 23108

Diego Patzan - 23525

**GUATEMALA, 26 de octubre del 2026**

**Informe parcial**

**Detección automática de enfermedades en hojas de plantas mediante redes neuronales convolucionales**

https://github.com/UVG-Tareas-y-proyectos/proyecto_final_dl

## Descripción del problema

La identificación de enfermedades en hojas requiere experiencia y puede complicarse cuando distintas afecciones presentan síntomas parecidos. Proponemos una aplicación local que reciba una fotografía y muestre una clasificación sugerida, como una exploración del uso de Deep Learning en sanidad vegetal.

## Objetivos

Nuestro objetivo es comparar una CNN entrenada desde cero con MobileNetV2 preentrenada en ImageNet, utilizando las mismas particiones de PlantVillage. Prepararemos datos reproducibles, mediremos accuracy, precision, recall y F1, analizaremos la matriz de confusión e integraremos el modelo en una interfaz de carga de imágenes.

## Análisis de los datos

PlantVillage contiene aproximadamente 54 mil imágenes, 14 cultivos y 38 clases de cultivo y estado sanitario. Trabajaremos con la variante RGB color de la fuente original y conservaremos su atribución. La ficha oficial declara licencia CC BY-SA 3.0. El inventario efectivo dependerá de la revisión descargada y de los filtros aplicados.

En el primer piloto seleccionamos 20 hojas por clase, hasta dos fotos por hoja, de papa con tizón temprano, tizón tardío y hojas sanas. Obtuvimos 117 imágenes de 60 hojas. No encontramos archivos dañados, duplicados exactos ni metadatos de hoja faltantes en esta selección.

| Partición | Imágenes | Hojas | Uso |
|---|---|---|---|
| Entrenamiento | 81 | 42 | Ajustar pesos |
| Validación | 18 | 9 | Seleccionar época |
| Prueba | 18 | 9 | Evaluación final |

## Propuesta de solución

El flujo implementado prepara un manifiesto de imágenes, entrena una de las dos redes, selecciona un checkpoint mediante validación y evalúa posteriormente el conjunto de prueba. La interfaz Streamlit permite cargar un JPG o PNG y muestra la clase predicha y las puntuaciones softmax.

## Construcción de entrenamiento validación y prueba

Utilizamos leaf-map.json del proveedor para mantener todas las fotografías de una hoja en una sola partición. Agrupamos también los duplicados exactos por SHA-256 y excluimos las imágenes sin metadatos de hoja. El reparto aproximado por grupos de cada clase es 70/15/15 con semilla 42; los porcentajes de imágenes pueden variar. Cada clase requiere al menos tres hojas independientes.

Guardamos las rutas, etiquetas, grupos y hashes en un CSV. Antes de entrenar o evaluar, verificamos la integridad de las imágenes y que no existan hojas ni duplicados exactos compartidos entre particiones. Esto no garantiza detectar imágenes visualmente similares o errores del metadato original.

## Preprocesamiento de imágenes

Convertimos las imágenes a RGB y las redimensionamos a 224 x 224. Aplicamos normalización de ImageNet para ambos modelos. El flip horizontal y las rotaciones de hasta 15 grados se aplican solamente durante entrenamiento; validación, prueba e inferencia usan transformaciones deterministas.

## Arquitecturas y entrenamiento

La CNN utiliza tres bloques de convolución 3 x 3, ReLU y max-pooling, con 16, 32 y 64 canales; termina con agregación espacial, dropout de 0.2 y una capa clasificadora. MobileNetV2 conserva el extractor preentrenado congelado y sustituye su clasificador por una capa con el número de clases del manifiesto.

La configuración inicial utiliza AdamW, learning rate de 0.001 y batch de 32. Proponemos 15 épocas para la comparación completa y repeticiones con semillas 42, 43 y 44. La mejor época se selecciona por F1 macro de validación. Prueba no se utiliza para ajustar hiperparámetros.

## Herramientas aplicadas

Python, PyTorch y Torchvision para las redes; Pillow para las imágenes; scikit-learn para las métricas; Streamlit para la interfaz y GitHub para conservar el código. El procesamiento de las imágenes y la inferencia se realizan localmente.

**Resultados del primer piloto**

Entrenamos la CNN durante cinco épocas con 81 imágenes. La mejor época por F1 macro de validación fue la tercera, con 0.5556. Evaluamos ese checkpoint en las 18 imágenes de prueba y conservamos sus predicciones individuales para revisar errores.

| **Métrica** | **Resultado CNN** |
|---|---|
| Accuracy | 0.5000 |
| Precision macro | 0.3929 |
| Recall macro | 0.5000 |
| F1 macro | 0.4000 |
| Imágenes de prueba | 18 |

**Matriz de confusión**

Las filas representan la clase real y las columnas la clase predicha.

| **Real y predicha** | **Temprano** | **Tardío** | **Sana** |
|---|---|---|---|
| Tizón temprano | 6 | 0 | 0 |
| Tizón tardío | 3 | 3 | 0 |
| Sana | 5 | 1 | 0 |

La CNN acertó nueve de las 18 imágenes y no reconoció correctamente ninguna hoja sana. Esto indica que el entrenamiento del piloto todavía es insuficiente para separar las tres clases. Con solo seis imágenes por clase en prueba, el resultado verifica el flujo y constituye una línea base, pero no permite estimar el desempeño general.

**Verificación del programa**

Pasaron nueve pruebas automatizadas sobre particiones por hoja, integridad de imágenes, dimensiones de ambas redes, congelamiento del extractor, persistencia del checkpoint, conversión RGB e interfaz con y sin modelo. El entrenamiento y la evaluación de la CNN se ejecutaron con imágenes reales.

**Estado de MobileNetV2**

La arquitectura se verificó con pesos aleatorios en las pruebas de software. La descarga de pesos ImageNet fue bloqueada por el filtro de red del entorno. No se completó un entrenamiento con transferencia ni se reportan métricas de ese modelo; será necesario ejecutarlo en una red que permita acceder al servidor oficial de PyTorch.